

Performance Challenge

AI Engineering - Terminal Chatbot

Title:

Multi-Provider Terminal Chatbot with Memory, RAG and AI-Assisted Development

Use Case (Epic):

As a junior AI developer, build a terminal-based chatbot that can work with more than one LLM provider, preserve short-term conversation context, retrieve information from a document knowledge base, and leave a clear trace of how AI coding assistants were used during development.

You may use any programming language. The repository, documentation and terminal-facing messages must be in English.

Functional Requirements

1. Terminal interaction

- Run the chatbot from a terminal with one documented command.
- Accept multiple messages in the same session until the user exits.
- Show the active provider/model and provide a clear way to switch providers and clear memory.

2. Multi-provider AI integration

- Integrate the official OpenAI SDK and the official Anthropic SDK.
- Use a shared provider interface/adaptor so the conversation logic is not duplicated for each provider.
- The active provider must be selectable through configuration or a terminal command. OpenRouter or other compatible providers may be added, but they do not replace the two required SDK integrations.
- Store credentials in environment variables; never hardcode API keys.

3. Conversation memory and saved chats

- Maintain a rolling window of the 10 most recent user/assistant messages; when it exceeds 10, remove the oldest message.
- For every model call, send the system instructions plus only that 10-message window to the selected provider.
- Persist complete chat sessions locally across application restarts. SQLite is recommended; JSON is also acceptable.
- Provide `/memory` to inspect the active window, `/chats` to list saved sessions by stable ID, and `/resume <session-id>` to continue one. Resuming must rebuild active context from that session's last 10 messages and save new messages back to the session.

4. RAG over local documents

- Ingest at least 3 local documents such as PDF, Markdown or text files.
- Apply chunking with overlap before embeddings. Start around 500-800 tokens per chunk with 10-20% overlap and adjust to the document structure.
- Store embeddings in a vector database. Supabase with PostgreSQL + pgvector is the recommended starting option.
- Retrieve relevant chunks with source metadata before generation. If the context does not support an answer, say there is not enough information instead of inventing content.



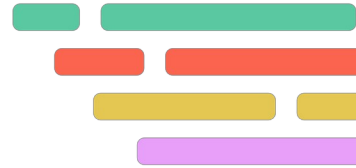
www.riwi.io



301 732 53 27



Cl. 16 # 55 - 129



5. AI-assisted development trace

- Create a /coding-assistance/ folder in the repository.
- coding-assistance/README.md must state which coding agent(s), model(s) and AI tools were used, and what they were used for.
- coding-assistance/prompts.jsonl must record every prompt sent to an AI coding assistant during development. Each entry should include at least: timestamp, agent/tool, model, prompt and purpose.
- Do not store API keys, tokens, passwords or other secrets in the log.

6. Custom skills

- Include at least 2 reusable custom skills in /skills/ or the equivalent folder used by the selected agent/framework.
- Each skill must document its purpose, expected input, expected output and how it is invoked.
- Both skills must be usable in a working demonstration. Examples include knowledge-base lookup, summarization, structured extraction or source explanation.

7. Configuration and failure handling

- Provide a .env.example containing every required environment variable without real credentials.
- Handle missing credentials, provider/API failures, empty retrieval results and unsupported documents with readable terminal messages.
- Recoverable errors must not terminate the application unexpectedly.
- README.md must explain setup, knowledge-base ingestion, how to run the chatbot, provider switching, saved-chat commands and how to test the RAG flow.

8. Deployed services

- The terminal chatbot may run locally, but every project service it consumes must be deployed and reachable through a documented URL; localhost-only service dependencies are not acceptable.
- Managed services such as OpenAI, Anthropic and hosted Supabase already qualify. Document all service URLs and required environment variables in README.md; never expose secrets.

Suggested Starting Guide

1. Define one common provider contract and normalize messages into a provider-independent format.
2. Implement OpenAI and Anthropic adapters behind that contract.
3. Add persistent sessions (SQLite/JSON), 10-message memory, /memory, /chats and /resume <session-id>.
4. Build document ingestion: load -> chunk -> overlap -> embed -> store in the vector database.
5. Build retrieval: user query -> embedding -> top relevant chunks -> context added to the model request.
6. Add terminal commands, provider switching, skills and AI-prompt logging.
7. Deploy required services, document their URLs, then test both providers, resume and RAG.

Recommended RAG baseline:

Documents -> 500-800 token chunks -> 10-20% overlap -> embeddings -> vector search -> top 4-6 chunks -> model context

Recommended vector store: Supabase + pgvector. Local alternatives such as Chroma or Qdrant are also acceptable.



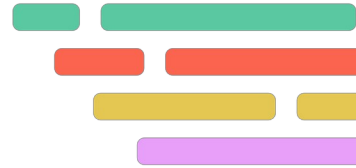
www.rivi.io



301 732 53 27



Cl. 16 # 55 - 129



Suggested libraries (examples, not mandatory):

OpenAI SDK + Anthropic SDK for providers; LangChain text splitters or LlamaIndex for chunking; Supabase client + pgvector for vector storage; dotenv/python-dotenv for configuration. Equivalent libraries are acceptable in other languages.

Expected Project Structure

```
src/{providers, memory, sessions, rag, skills, terminal} | data/{documents, chats.db} |
coding-assistance/{README.md, prompts.jsonl}
.env.example | README.md
```

Expected Deliverables

Component	What it must contain	Verification
Source code	Terminal chatbot with provider adapters, 10-message memory, persistent saved chats, RAG and skills.	Runs from the documented command; /chats and /resume work.
RAG pipeline	Ingestion, chunking, overlap, embeddings, vector storage and retrieval.	Relevant chunks return with source metadata.
Custom skills	At least 2 documented reusable skills.	Both work in a demo.
AI trace	coding-assistance/README.md and complete prompts.jsonl.	Prompts show tool/agent and model.
README + env	Setup, run, ingestion, provider switching, saved-chat commands, deployed service URLs, test steps and .env.example.	A new user can configure the local CLI to use the deployed services without real secrets.

Acceptance Criteria

- The project runs from the terminal by following the README.
- The same chat flow works with OpenAI and Anthropic without changing the core conversation logic.
- Saved chats survive an application restart; /chats lists them and /resume <session-id> continues one, while active model context remains capped at the 10 most recent user/assistant messages.
- RAG returns relevant chunks with source metadata and the chatbot does not invent unsupported answers.
- At least 2 custom skills are implemented and can be invoked in a working demo.
- Every AI-assisted development prompt is recorded in coding-assistance/prompts.jsonl with the agent/tool and model used.
- No API key or secret is hardcoded or committed; recoverable provider, retrieval or configuration errors do not crash the app.
- Every project service consumed by the chatbot is deployed and documented by URL; only the terminal client may run locally.
- The complete deliverable is written in English.



www.riwi.io



301 732 53 27



Cl. 16 # 55 - 129